# Data Mining Assignment 1 Report

Author: Juiyun Chang (314551145)

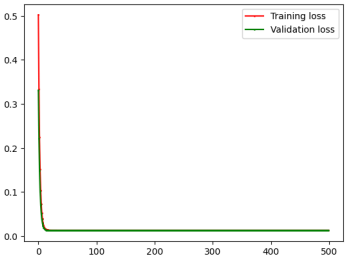
Github Link: https://github.com/pinkypacman/DM2026-Assignment-1

## 1. Linear Regression

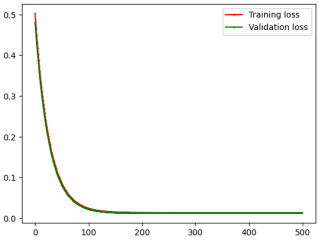
## (1a) Change the learning rate and plot the loss curve for each learning rate

### Linear Dataset A

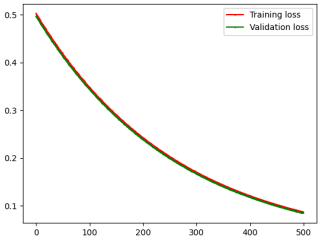
- lr = 0.1




- lr=0.01

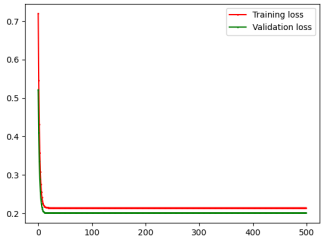



- lr = 0.001

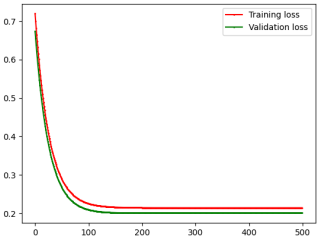



### Linear Dataset B

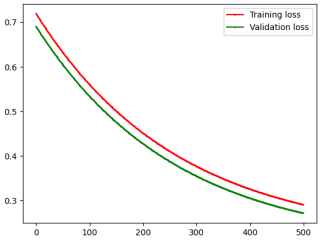
- lr = 0.1




- lr = 0.01

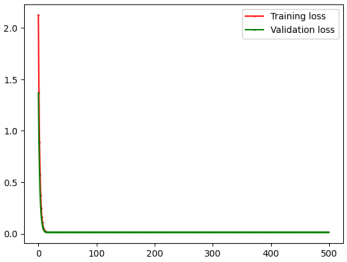



- lr = 0.001

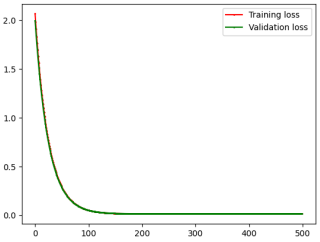



### Linear Dataset C

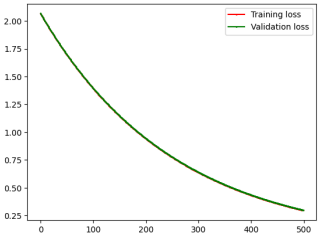
- lr = 0.1




- lr = 0.01

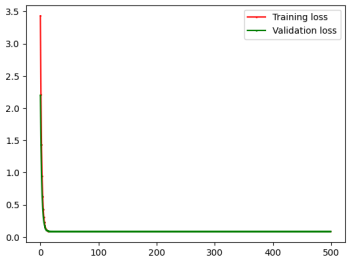



- lr = 0.001

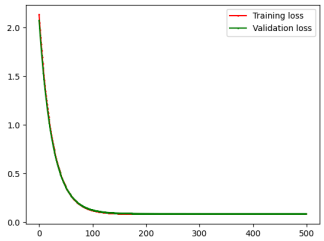


### Linear Dataset D

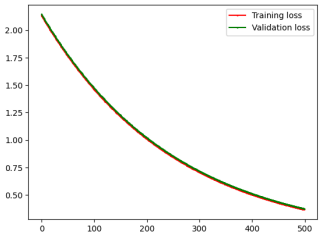
- lr = 0.1




- lr = 0.01




- lr = 0.001




## (1b) Evaluations using “evaluate_linear_regression” function

### Linear Dataset A

- lr=0.1

  MSE : 0.0134
  MAE : 0.1010
  RMSE : 0.1156
  R-squared : 0.5690

- lr=0.01

  MSE : 0.0134
  MAE : 0.1009
  RMSE : 0.1156
  R-squared : 0.5690

- lr=0.001

  MSE : 0.0832
  MAE : 0.2369
  RMSE : 0.2884
  R-squared : -1.6836

### Linear Dataset B

- lr=0.1

  MSE : 0.2137
  MAE : 0.4038
  RMSE : 0.4623
  R-squared : 0.0448

- lr=0.01

  MSE : 0.2137
  MAE : 0.4038
  RMSE : 0.4623
  R-squared : 0.0448

- lr=0.001

  MSE : 0.2687
  MAE : 0.4404
  RMSE : 0.5184
  R-squared : -0.2011

### Linear Dataset C

- lr=0.1

  MSE : 0.0132
  MAE : 0.0987
  RMSE : 0.1148
  R-squared : 0.9954

- lr=0.01

  MSE : 0.0132
  MAE : 0.0987
  RMSE : 0.1148
  R-squared : 0.9954

- lr=0.001

  MSE : 0.2852
  MAE : 0.4273
  RMSE : 0.5341
  R-squared : 0.8994

### Linear Dataset D

- lr=0.1

  MSE : 0.0824
  MAE : 0.2467
  RMSE : 0.2870
  R-squared : 0.9716

- lr=0.01

  MSE : 0.0824
  MAE : 0.2467
  RMSE : 0.2870
  R-squared : 0.9716

- lr=0.001

  MSE : 0.3540
  MAE : 0.4778
  RMSE : 0.5950
  R-squared : 0.8780

# (1c) Observations on Results of Linear Regression

## Linear Dataset A & B

- lr = 0.1

  The loss drops very quickly and converges almost immediately. Training remains stable despite the high learning rate, with no noticeable oscillation.

  In Dataset A, this leads to strong performance (low MSE, $R^2 \approx 0.57$). In Dataset B, although convergence is equally fast, the model plateaus at a higher loss due to the more scattered data distribution, resulting in lower overall performance.

  Overall, this learning rate is fast and effective, but performance is ultimately limited by data characteristics.

---

- lr = 0.01

  The loss decreases smoothly and steadily, with slightly slower convergence compared to lr = 0.1.

  In both datasets, the final performance is nearly identical to lr = 0.1, indicating that a slightly smaller learning rate does not harm optimization.

  This setting provides a more conservative and stable training process while achieving comparable results.

---

- lr = 0.001
  The loss decreases very slowly due to the small learning rate, and the model remains underfitted after 500 iterations.

  In both datasets, evaluation metrics are significantly worse (higher errors and negative $R^2$), indicating ineffective learning within the given training budget.

  This suggests the learning rate is too small; however, increasing the number of iterations could potentially improve performance.

## Linear Dataset C & D

- **lr = 0.1**

  The loss drops very quickly and converges almost immediately, with stable training behavior and no oscillation.

  In both Dataset C and D, this leads to **excellent performance**, with very low error and near-perfect fit ($R^2 \approx 0.99$ for C, $\approx 0.97$ for D).

  This indicates that the model can easily capture the underlying linear relationship, and a high learning rate does not harm stability in this case.

---

- **lr = 0.01**

  The loss decreases smoothly and steadily, with slightly slower convergence compared to lr = 0.1.

  However, the final performance is **essentially identical** to lr = 0.1 in both datasets.

This suggests that the optimization landscape is simple, and both learning rates are sufficient to reach the global optimum.

This setting provides a more conservative and stable training process.

---

- **lr = 0.001**

  The loss decreases much more slowly, and the model is still not fully converged after 500 iterations.

  Although the performance is still relatively good (positive $R^2$), it is noticeably worse than the other two settings, with higher MSE, MAE, and RMSE.

  This indicates **underfitting due to insufficient training progress**, rather than model limitation. Increasing the number of iterations would likely improve results.

**Overall Insight**

- lr = 0.1 and 0.01 both achieve **near-optimal performance**, showing that optimization is not difficult in these datasets.
- lr = 0.001 is too small and slows convergence significantly.

---

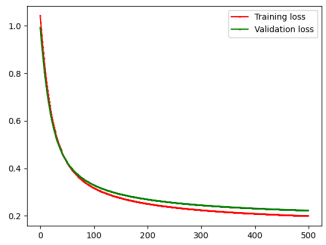
# 2. Logistic Regression

# (2a) Loss Curve Under Different Learning Rates

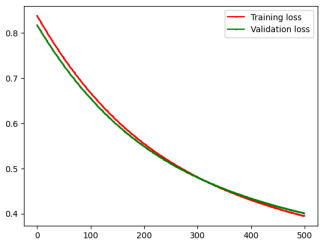
- FIXED n_iterations = 500

## Logistic Dataset A

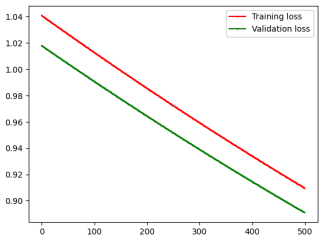
- lr=0.1




- lr=0.01

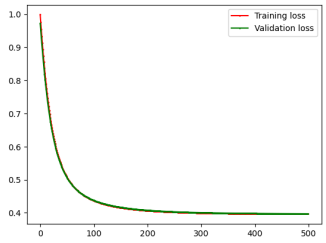



- lr=0.001

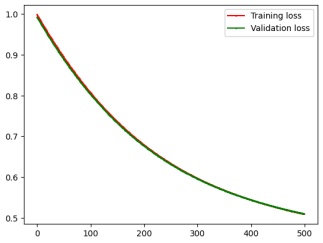



## Logistic Dataset B

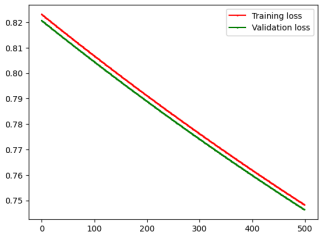
- lr=0.1




- lr=0.01

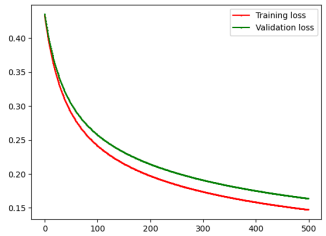



- lr=0.001

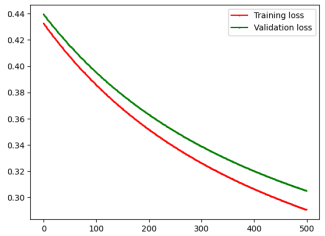


## Logistic Dataset C

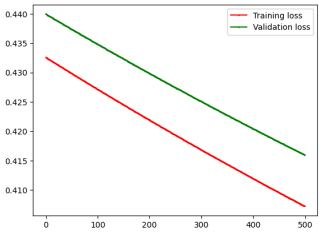
- lr=0.1




- lr=0.01

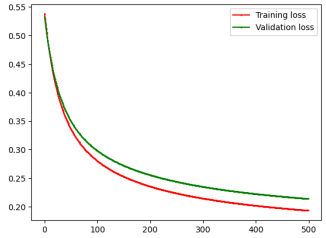



- lr=0.001

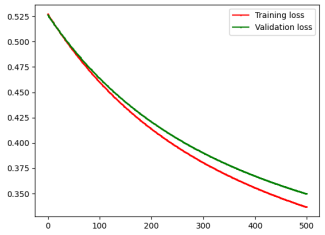



## Logistic Dataset D

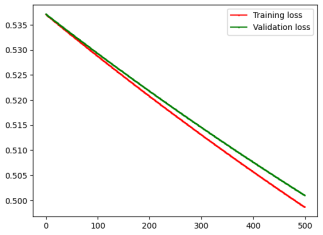
- lr=0.1




- lr=0.01




- lr=0.001

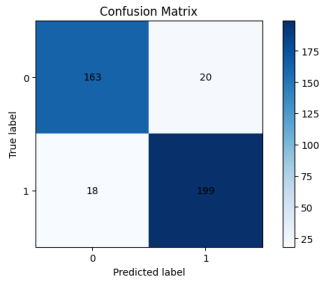



# (2b) Evaluation using "evaluate_binary_classifier" function

- FIXED n_iterations = 500

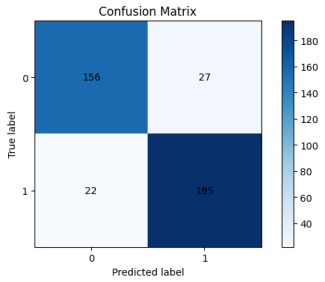
## Logistic Dataset A

- lr = 0.1



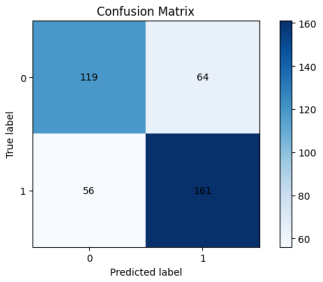

Accuracy : 0.9050
Precision : 0.9087
Recall : 0.9171
F1-score : 0.9128

- lr=0.01




Accuracy : 0.8775
Precision : 0.8784
Recall : 0.8986
F1-score : 0.8884

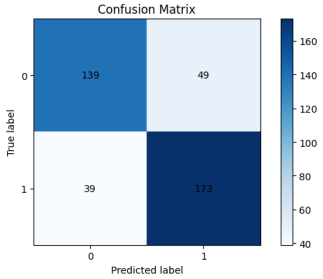
- lr=0.001



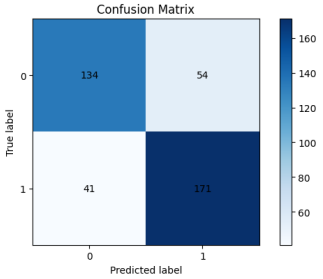

Accuracy : 0.3875
Precision : 0.4364
Recall : 0.4424
F1-score : 0.4394

## Logistic Dataset B

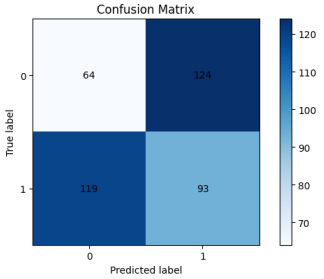
- lr=0.1
- lr=0.01
- lr=0.001




Accuracy : 0.7800
Precision : 0.7793
Recall : 0.8160
F1-score : 0.7972



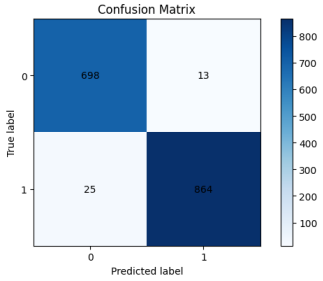

Accuracy : 0.7625
Precision : 0.7600
Recall : 0.8066
F1-score : 0.7826




Accuracy : 0.3925
Precision : 0.4286
Recall : 0.4387
F1-score : 0.4336

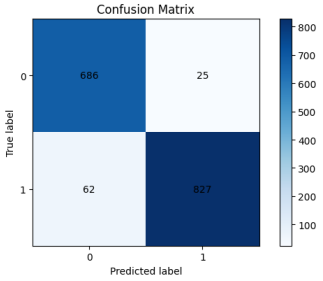
## Logistic Dataset C

- lr=0.1



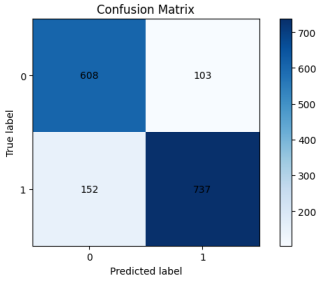

Accuracy : 0.9762
Precision : 0.9852
Recall : 0.9719
F1-score : 0.9785

- lr=0.01




Accuracy : 0.9456
Precision : 0.9707
Recall : 0.9303
F1-score : 0.9500

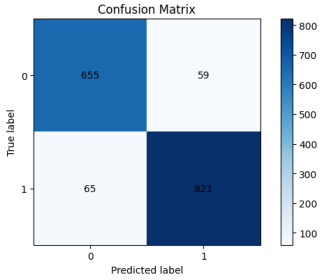
- lr=0.001




Accuracy : 0.8406
Precision : 0.8774
Recall : 0.8290
F1-score : 0.8525

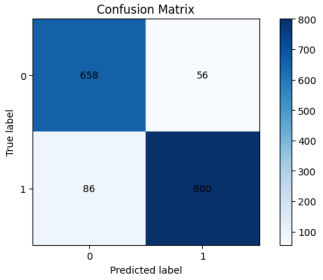
## Logistic Dataset D

- lr=0.1



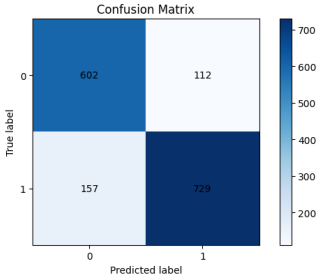

Accuracy : 0.9225
Precision : 0.9330
Recall : 0.9266
F1-score : 0.9298

- lr=0.01




Accuracy : 0.9113
Precision : 0.9346
Recall : 0.9029
F1-score : 0.9185

- lr=0.001




Accuracy : 0.8319
Precision : 0.8668
Recall : 0.8228
F1-score : 0.8442

## (2c) Observation on 2a and 2b

From the overall trend, we can see that the Learning rate strongly controls convergence speed and final performance.

- lr = 0.1 → best overall performance

  A higher learning rate (lr = 0.1) leads to rapid loss reduction and achieves the best overall metrics, with training and validation curves closely aligned, indicating good generalization.

- lr = 0.01 → stable but slightly worse

  A moderate learning rate (lr = 0.01) shows smoother and more gradual convergence, reaching slightly worse but still competitive performance, making it a stable and conservative alternative.

- lr = 0.001 → severe underfitting (consistently poor results)

  A small learning rate (lr = 0.001) results in slow, nearly linear loss decay and consistently poor performance across all datasets, suggesting that the model fails to converge within the fixed 500 iterations and suffers from underfitting.

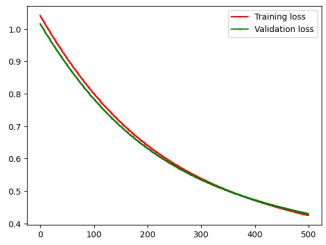
This effect is more pronounced in the noisier datasets (A and B), where performance drops significantly at lr = 0.001, indicating that more aggressive updates are needed to learn meaningful decision boundaries. In the easier datasets (C and D), the gap between lr = 0.1 and lr = 0.01 is smaller, although lr = 0.1 still achieves the best results. Overall, these results show that, under a limited training budget, choosing a sufficiently large learning rate is crucial for effective learning, while an overly small learning rate can severely hinder model performance.

## (2d) Training under different iterations with a fixed learning rate

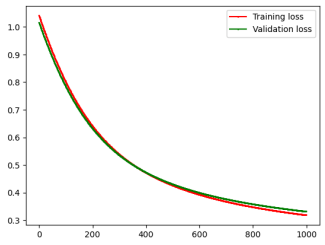
- Fixed learning rate = 0.01

### Logistic Dataset A

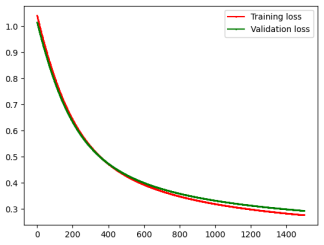
- n_iteration = 500




- n_iteration = 1000

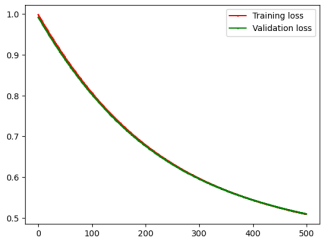


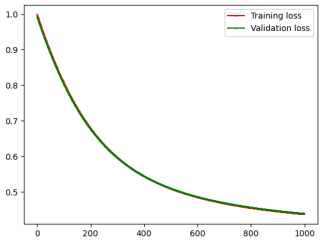

- n_iteration = 1500

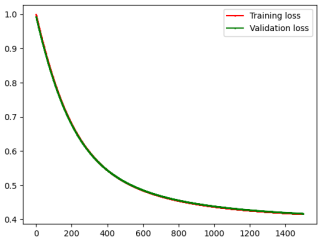



### Logistic Dataset B

- n_iteration = 500
- n_iteration = 1000
- n_iteration = 1500

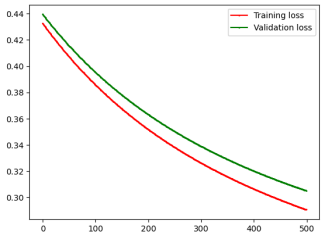


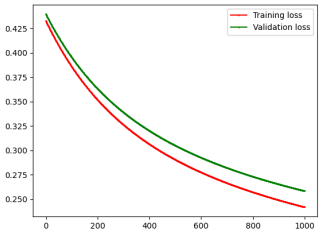




## Logistic Dataset C

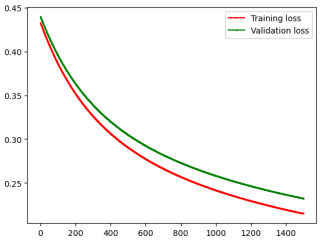
- n_iteration = 500




- n_iteration = 1000

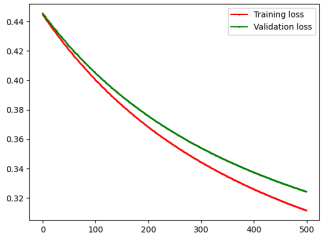



- n_iteration = 1500

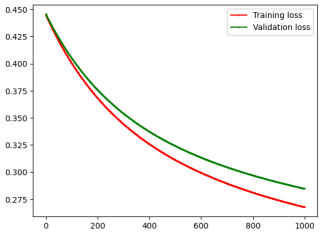



## Logistic Dataset D

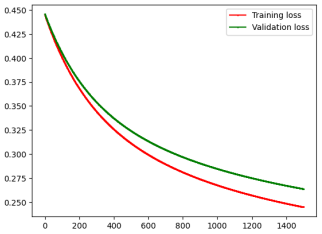
- n_iteration = 500




- n_iteration = 1000



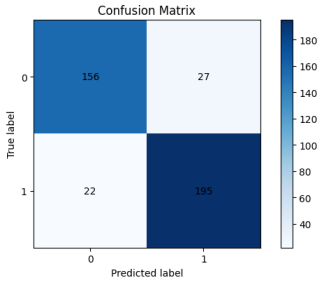

- n_iteration = 1500




# (2e) Evaluation using “evaluate_binary_classifier” function

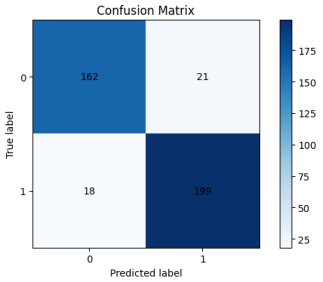
## Logistic Dataset A

- n_iteration = 500



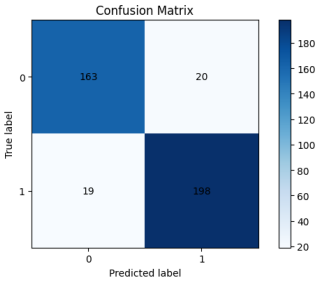

Accuracy : 0.8775
Precision : 0.8784

- n_iteration = 1000




Accuracy : 0.9025
Precision : 0.9045

- n_iteration = 1500




Accuracy : 0.9025
Precision : 0.9083

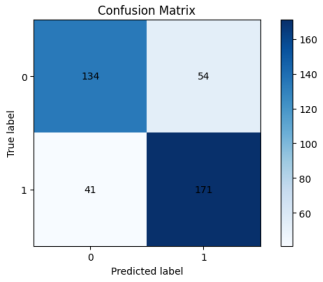
Recall : 0.8986
F1-score : 0.8884

Recall : 0.9171
F1-score : 0.9108

Recall : 0.9124
F1-score : 0.9103

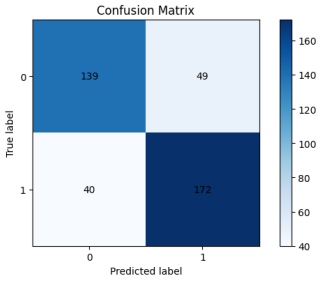
## Logistic Dataset B

- n_iteration = 500



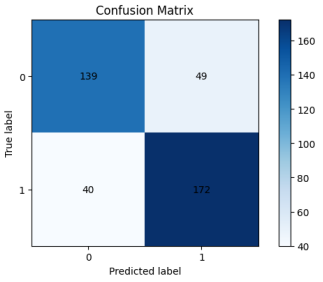

Accuracy : 0.7625
Precision : 0.7600
Recall : 0.8066
F1-score : 0.7826

- n_iteration = 1000




Accuracy : 0.7775
Precision : 0.7783
Recall : 0.8113
F1-score : 0.7945

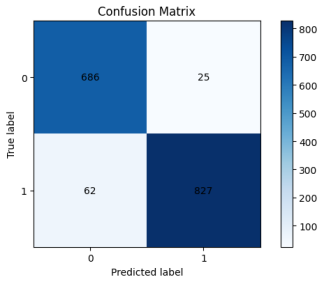
- n_iteration = 1500




Accuracy : 0.7775
Precision : 0.7783
Recall : 0.8113
F1-score : 0.7945

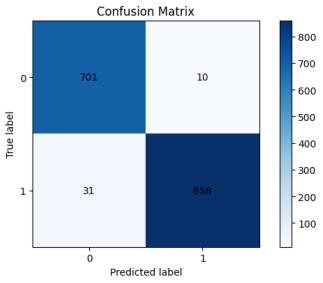
## Logistic Dataset C

- n_iteration = 500



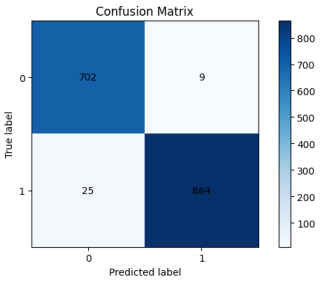

Accuracy : 0.9456
Precision : 0.9707
Recall : 0.9303
F1-score : 0.9500

- n_iteration = 1000




Accuracy : 0.9744
Precision : 0.9885
Recall : 0.9651
F1-score : 0.9767

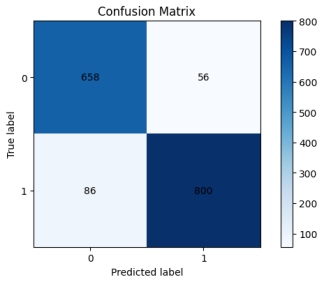
- n_iteration = 1500

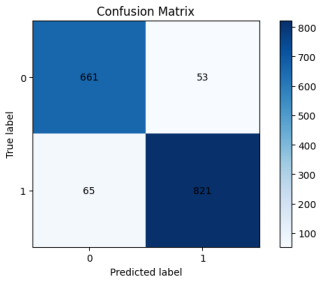


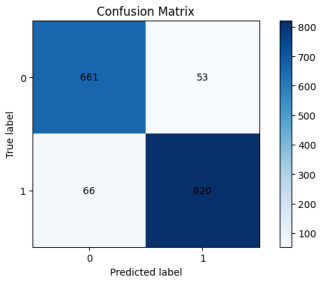

Accuracy : 0.9788
Precision : 0.9897
Recall : 0.9719
F1-score : 0.9807

## Logistic Dataset D

- n_iteration = 500
- n_iteration = 1000
- n_iteration = 1500




Accuracy : 0.9113
Precision : 0.9346
Recall : 0.9029
F1-score : 0.9185

Accuracy : 0.9263
Precision : 0.9394
Recall : 0.9266
F1-score : 0.9330

Accuracy : 0.9256
Precision : 0.9393
Recall : 0.9255
F1-score : 0.9323

## (2f) Observation on 2d and 2e

With a fixed learning rate (lr = 0.01), training across different iteration counts shows clear improvement as iterations increase, but with diminishing returns. Early on, adding iterations helps further reduce loss and improves performance, indicating that the model is still converging. Beyond a certain point, the model nears its optimum, and additional training yields only small gains, highlighting the trade-off between computational cost and performance.

- n_iteration = 500
  The model has not fully converged, as both training and validation loss are still decreasing. Performance metrics are lower, indicating mild underfitting.
- n_iteration = 1000
  The model moves closer to convergence, with further loss reductions and noticeable improvements in accuracy, precision, recall, and F1-score.
- n_iteration = 1500
  The model is largely converged; loss decreases only slightly, and performance gains are marginal, suggesting limited benefit from additional iterations.

---

# 3. Data Preprocessing

## (3a, 3b) Median & Standard Deviation for columns with missing values

- Before Imputation

| | SepalLengthCm | SepalWidthCm | PetalLengthCm | PetalWidthCm | BranchLength |
|---|---|---|---|---|---|
| median | 6.300000 | 2.900000 | 5.085612 | 1.600000 | 16.300000 |
| std | 1.037115 | 0.389564 | 1.582807 | 0.706689 | 1.035228 |

- After Imputation

| | SepalLengthCm | SepalWidthCm | PetalLengthCm | PetalWidthCm | BranchLength |
|---|---|---|---|---|---|
| median | 6.300000 | 2.900000 | 5.035683 | 1.700000 | 16.300000 |

| | | | | | |
|---|---|---|---|---|---|
| std | 1.009275 | 0.372434 | 1.514955 | 0.694645 | 1.011012 |

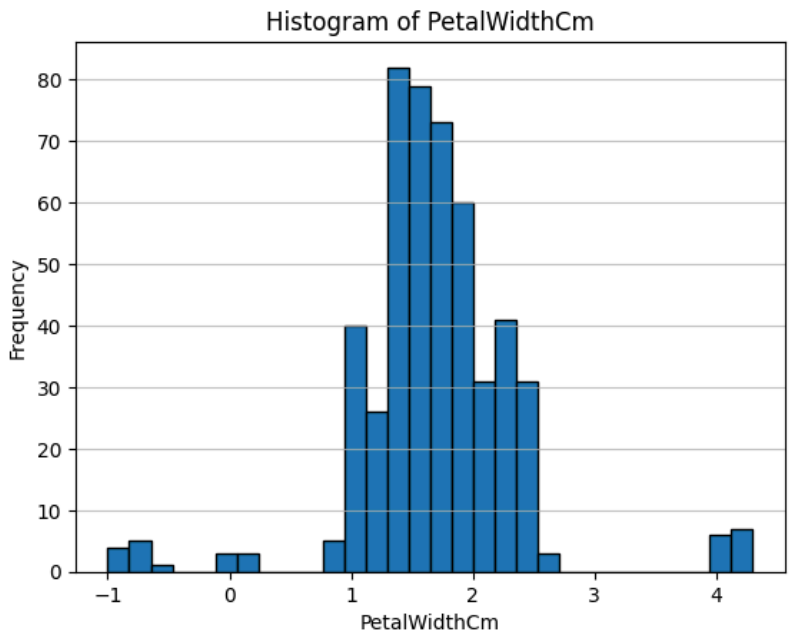
- Observation on the difference between the median and standard deviation before and after KNN imputation

  KNN imputes missing values using nearby samples, so it preserves the central tendency well. The results above show that KNN imputation largely maintains the median across features, indicating that the central distribution remains stable after imputation. However, the standard deviation decreases slightly for all features, suggesting a mild smoothing effect as missing values are replaced with averages from neighboring samples. This reduces variability while maintaining the overall structure of the dataset.

---

## 4. Data Exploration

## (4a) Plot the histogram of the column “PetalWidthCm.”




The histogram of PetalWidthCm shows a roughly unimodal distribution, with most values concentrated between about 1.0 and 2.5 and peaking around 1.5–2.0. The distribution is slightly right-skewed, with a longer tail toward higher values. A few outliers appear at both the lower and upper ends, but overall the data remain centered and well-structured.

## (4b) List the feature (excluding “PetalWidthCm”, “Species”, and “Id”) that has the largest positive correlation (Pearson correlation coefficient) with the “PetalWidthCm” column.

```
Correlation values:
 PetalWidthCompactness    0.991670
PetalWidthElongation     0.991551
PetalWidthCurvature      0.991273
PetalWidthMajorAxis      0.991141
PetalWidthSpread         0.991095
                           ...
SepalWidthMinorAxis     -0.074392
SepalWidthCurvature     -0.081334
SepalWidthCompactness   -0.088496
SepalGlossIndex         -0.095200
SepalWidthMajorAxis     -0.096350
```

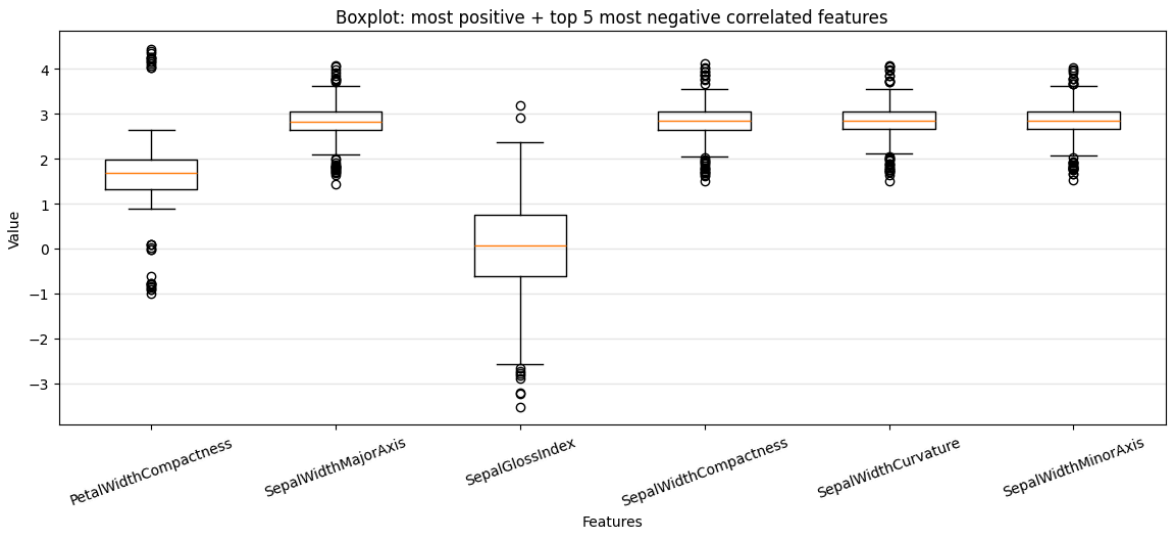
- The most correlated feature is "**PetalWidthCompactness**"

## (4c) Report the top 5 features with the strongest negative correlations with the 'PetalWidthCm' column.

- Top 5 most correlated features with PetalWidthCm

| Rank | Feature | Correlation |
|---|---|---|
| 1 | SepalWidthMajorAxis | -0.096350 |
| 2 | SepalGlossIndex | -0.095200 |
| 3 | SepalWidthCompactness | -0.088496 |
| 4 | SepalWidthCurvature | -0.081334 |
| 5 | SepalWidthMinorAxis | -0.074392 |

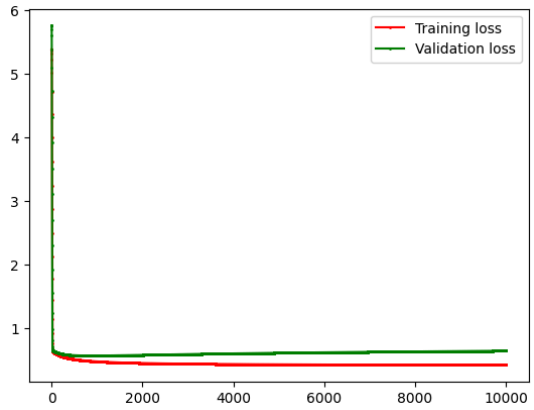
## (4d) Plot the boxplot for all features found in 4(b) and 4(c).



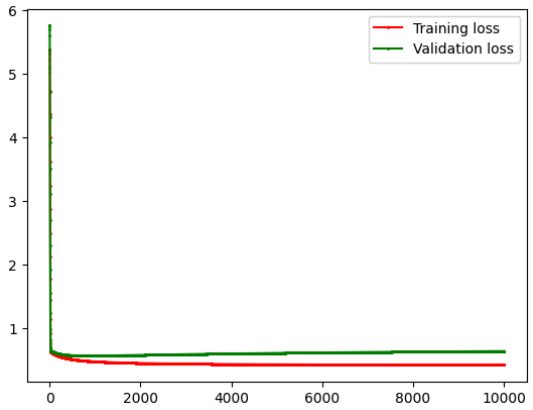
# 5. Regularization

## (5a) L2 Regularization

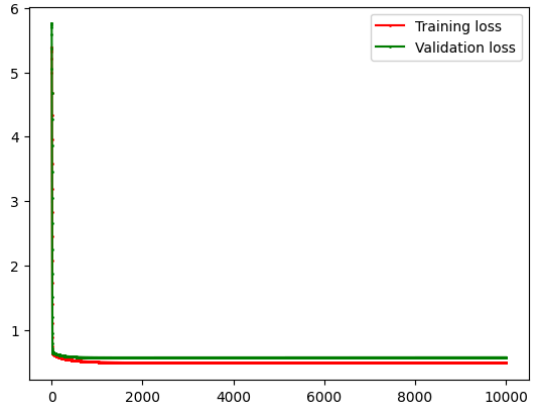
- No Regularization



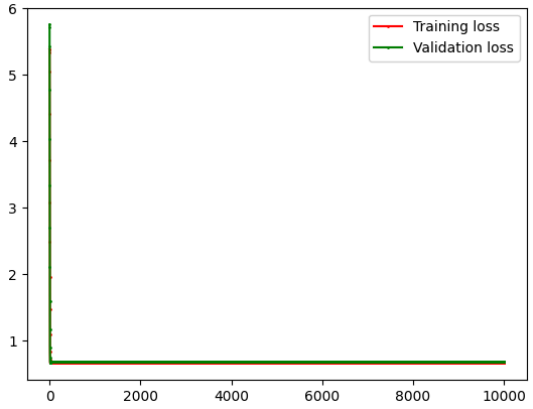

- L2 (reg_lambda=0.01)




- L2 (reg_lambda=1)

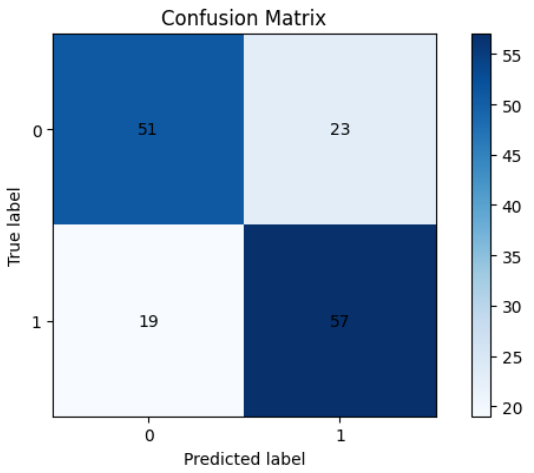



- L2 (reg_lambda=100)

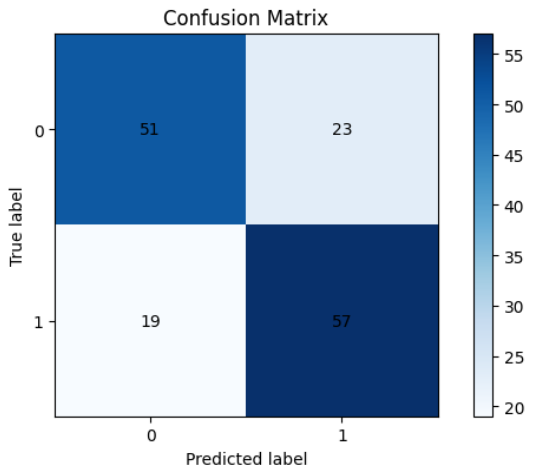



## (5b) Testing results using the "evaluate_binary_classifier" function.

- No Regularization
- L2 (reg_lambda=0.01)

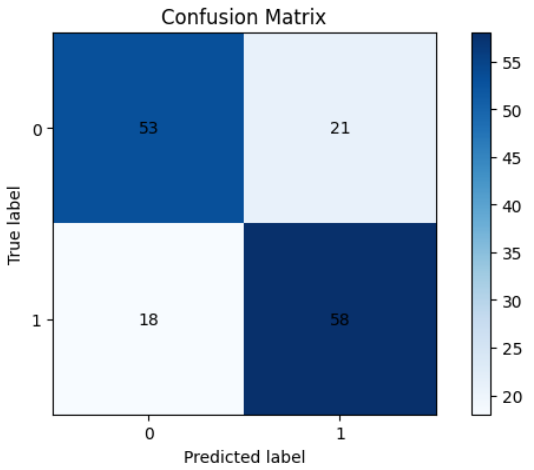



Accuracy : 0.7200
Precision : 0.7125
Recall : 0.7500
F1-score : 0.7308



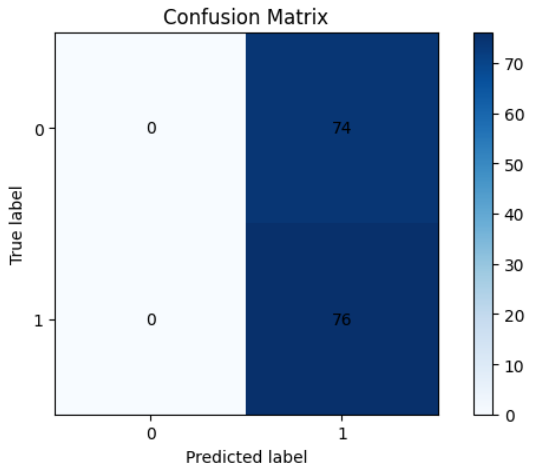

Accuracy : 0.7200
Precision : 0.7125
Recall : 0.7500
F1-score : 0.7308

- L2 (reg_lambda=1)




Accuracy : 0.7400
Precision : 0.7342
Recall : 0.7632
F1-score : 0.7484

- L2 (reg_lambda=100)




Accuracy : 0.5067
Precision : 0.5067
Recall : 1.0000
F1-score : 0.6726

## (5c) Observations in 5(a) and 5(b).

As the L2 regularization strength increases, the model becomes more constrained, affecting both training behavior and final performance:

- **λ = 0 / 0.01 (No or very weak regularization)**
  Training loss converges quickly to a low value, and testing performance is identical (Accuracy ≈ 0.72).

→ Regularization has **minimal effect**, and the model fits the data well but without noticeable generalization improvement.

- **λ = 1 (Moderate regularization)**
  Training loss is slightly higher, but convergence remains stable. Testing performance improves (Accuracy ≈ 0.74, higher F1-score).
  → This indicates a **better balance between fitting and generalization**, reducing overfitting.
- **λ = 100 (Strong regularization)**
  Training loss remains high due to heavy constraints. Testing performance drops significantly (Accuracy ≈ 0.51), with Recall = 1.0 but low Precision.
  → The model predicts almost all samples as positive, showing **severe underfitting and bias**.

**Overall Insight**

- Small λ: little to no impact
- Moderate λ: **best performance (improved generalization)**
- Large λ: **over-regularization → underfitting and poor results**